UNIT 5

INTRODUCTION TO CONTROL FLOW GRAPGH

LECTURE NO:1

DEFINITION OF CONTROL FLOW GRAPGH

In a control-flow graph each node in the graph represents a basic block, i.e. a straight-line piece of code without any jumps or jump targets; **jump targets start a block, and jumps end a block**. Directed edges are used to represent jumps in the control flow.

Basic Blocks and Flow Graphs

Basic Block

The basic block is a set of statements. The basic blocks do not have any in and out branches except entry and exit. It means the flow of control enters at the beginning and will leave at the end without any halt. The set of instructions of basic block executes in sequence.

Here, the first task is to partition a set of three-address code into the basic block. The new basic block always starts from the first instruction and keep adding instructions until a jump or a label is met. If no jumps or labels are found, the control will flow in sequence from one instruction to another.

The algorithm for the construction of basic blocks is given below:

Algorithm: Partitioning three-address code into basic blocks.

Input: The input for the basic blocks will be a sequence of three-address code.

Output: The output is a list of basic blocks with each three address statements in exactly one block.

METHOD: First, we will identify the leaders in the intermediate code. There are some rules for finding leaders, which are given below:

1. The first instruction in the intermediate code will always be a leader.

2. The instructions that target a conditional or unconditional jump statement are termed as a leader.

3. Any instructions that are just after a conditional or unconditional jump statement will be a leader.

Each leader's basic block will have all the instructions from the leader itself until the instruction, which is just before the starting of the next leader.

Example:

```
1
3   for i from 1 to 10 do
4   for j from 1 to 10 do
    a [ i, j ] = 0.0;
5   for i from 1 to 10 do
    a [ i,i ] = 1.0;
6
7
```

```
1
2   1) i = 1
3   2) j = 1
    3) t1 = 10 * i
4   4) t2 = t1 + j
    5) t3 = 8 * t2
5   6) t4 = t3 - 88
6   7) a[t4] = 0.0
    8) j = j + 1
7   9) if j <= 10 goto (3)
    10) i = i + 1
8   11) if i <= 10 goto (2)
9   12) i = 1
    13) t5 = i - 1
10  14) t6 = 88 * t5
    15) a[t6] = 1.0
11  16) i = i + 1
    17) if i <= 10 goto (13)
12
```

- According to the given algorithm, instruction 1 is a leader.

- Instruction 2 is also a leader because this instruction is the target for instruction 11.
- Instruction 3 is also a leader because this instruction is the target for instruction 9.
- Instruction 10 is also a leader because it immediately follows the conditional goto statement.
- Similar to step 4, instruction 12 is also a leader.
- Instruction 13 is also a leader because this instruction is the target for instruction 17.

So there are six basic blocks for the above code, which are given below:

B1 for statement 1

B2 for statement 2

B3 for statement 3-9

B4 for statement 10-11

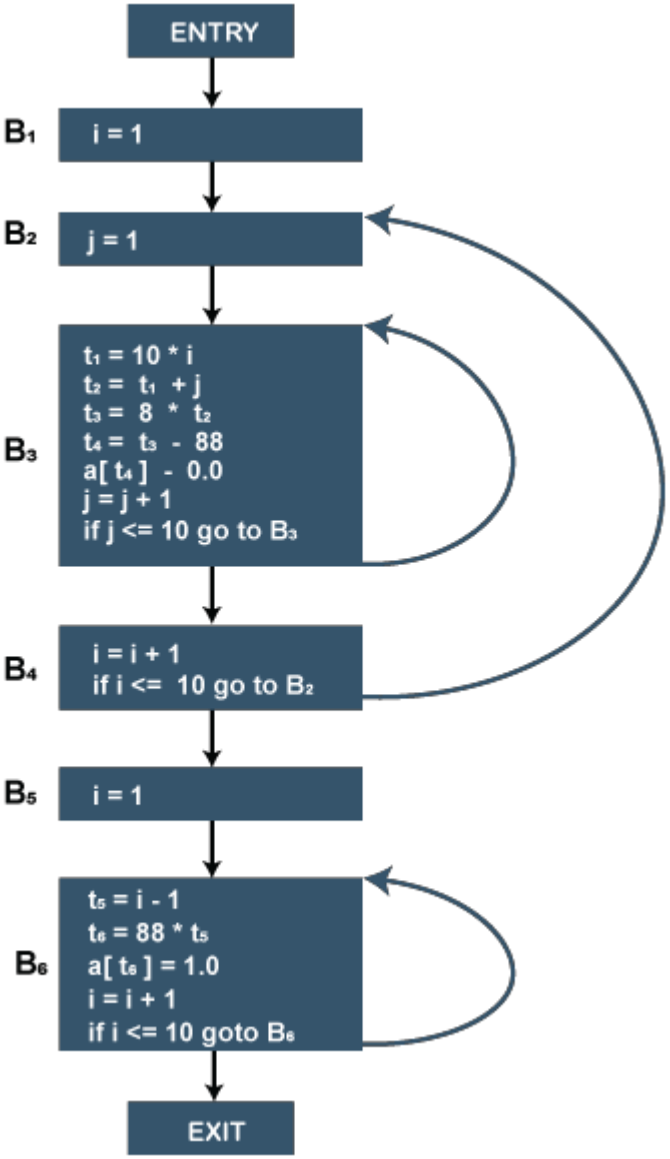
B5 for statement 12

B6 for statement 13-17.

Flow Graph

It is a directed graph. After partitioning an intermediate code into basic blocks, the flow of control among basic blocks is represented by a flow graph. An edge can flow from one block X to another block Y in such a case when the Y block's first instruction immediately follows the X block's last instruction. The following ways will describe the edge:

- There is a conditional or unconditional jump from the end of X to the starting of Y.
- Y immediately follows X in the original order of the three-address code, and X does not end in an unconditional jump.



Flow graph for the 10 x 10 matrix to an identity matrix.

- Block B1 is the entry point for the flow graph because B1 contains starting instruction.
- B2 is the only successor of B1 because B1 doesn't end with unconditional jumps, and the B2 block's leader immediately follows the B1 block's leader.
- B3 block has two successors. One is a block B3 itself because the first instruction of the B3 block is the target for the conditional jump in the last instruction of block B3. Another successor is block B4 due to conditional jump at the end of B3 block.
- B6 block is the exit point of the flow graph.

THE DAG REPRESENTATION FOR BASIC BLOCKS

WHAT IS DAG

- A DAG for a basic block is a directed acyclic graph with the following labels on nodes:

    1. Leaves are labelled by unique identifiers, either variable names or constants.
    2. Interior nodes are labelled by an operator symbol.

    3. Nodes are also optionally given a sequence of identifiers for labels to store the computed values.

- DAGs are useful data structures for implementing transformations on basic blocks.
- It gives a picture of how the value computed by a statement is used in subsequent statements.
- It provides a good way of determining common sub - expressions.

Algorithm for construction of DAG

Input: A basic block
Output: A DAG for the basic block containing the following information:

1. A label for each node. For leaves, the label is an identifier. For interior nodes, an operator symbol.

2. For each node a list of attached identifiers to hold the computed values. Case (i) x : = y OP z Case (ii) x : = OP y
Case (iii) x : = y

Method:
Step 1:
If y is undefined then create node(y).
If z is undefined, create node(z) for case(i).

Step 2:

For the case(i), create a node(OP) whose left child is node(y) and right child is node(z). (Checking for common sub expression). Let n be this node.

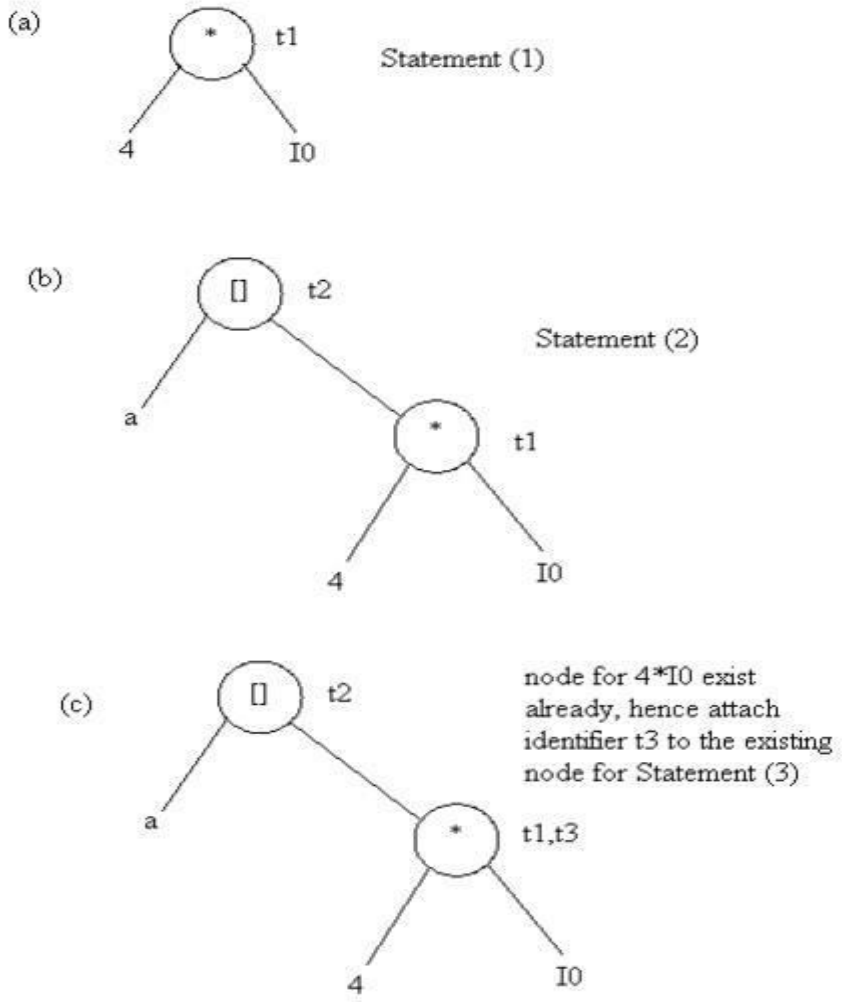
For case(ii), determine whether there is node(OP) with one child node(y). If not create such a node.
For case(iii), node n will be node(y).

Step 3:

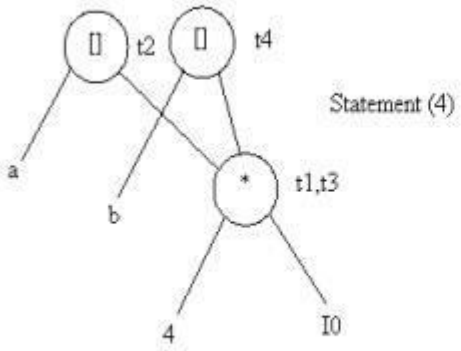
Delete x from the list of identifiers for node(x). Append x to the list of attached identifiers for the node n found in step 2 and set node(x) to n.

Example: Consider the block of three- address statements in Fig 4.6

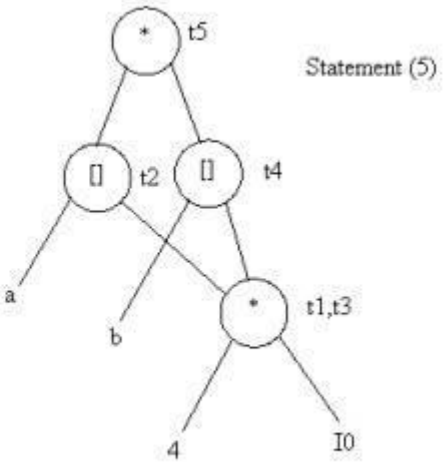
Stages in DAG Construction



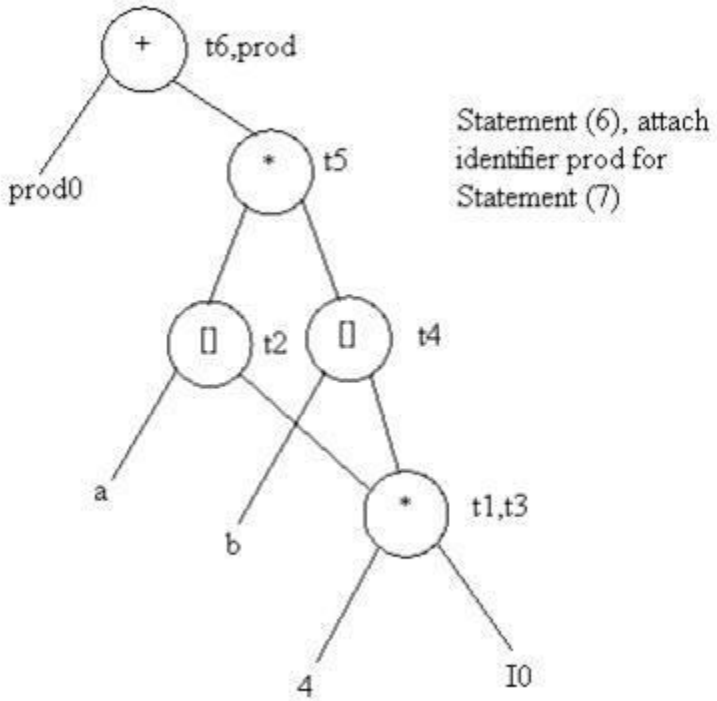
(d)


[] t2
[] t4
Statement (4)
a
b
*
t1,t3
4
I0

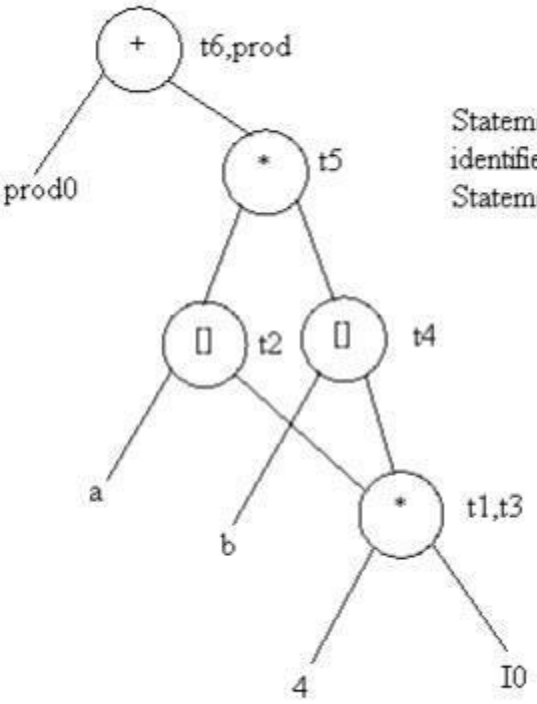
(e)


*
t5
Statement (5)
[] t2
[] t4
a
b
*
t1,t3
4
I0

(f)

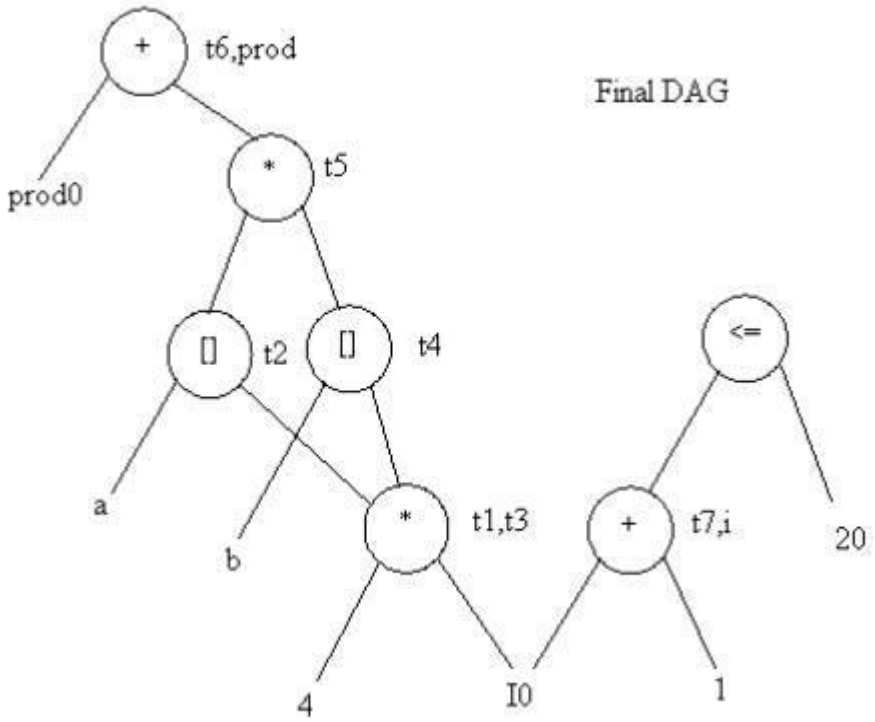

+
t6,prod
prod0
*
t5
Statement (6), attach identifier prod for Statement (7)
[] t2
[] t4
a
b
*
t1,t3
4
I0

(f)

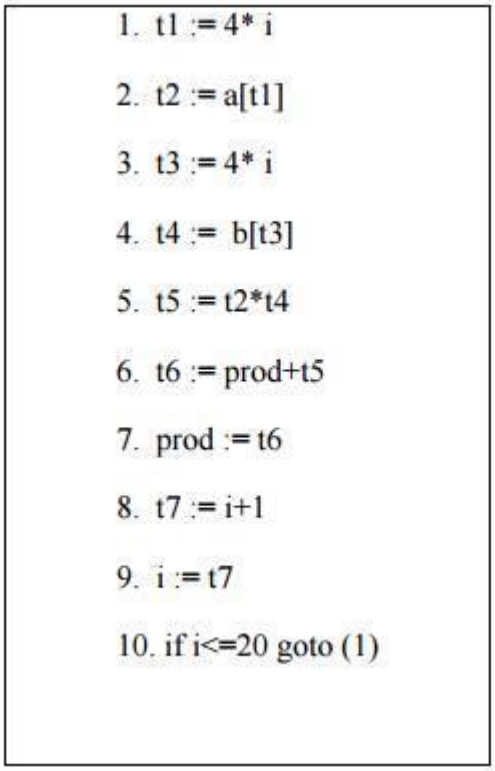


Statement (6), attach identifier prod for Statement (7)

(h)



Final DAG


1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)

Code Block

LECTURE NO:2
DAG ADVANTAGE

Application of DAGs:

1. We can automatically detect common sub expressions.
2. We can determine which identifiers have their values used in the block.
3. We can determine which statements compute values that could be used outside the block.

Advantages of using DAG technology

- Speed, perhaps its greatest advantage, unlike block chains the more transactions it has to process its response speed will be faster.
- Higher level of scalability, by not being subject to limitations on block creation times, a greater number of transactions can be processed than those processed by block chain networks. This is particularly attractive in the application of the Internet of Things.
- It does not require mining, its carbon footprint is a tiny fraction of that left by cryptocurrencies that require mining to generate their block chain.
- It does not generate commissions or transaction fees, since it does not require mining work and does not generate costs for the transmission of transactions. Although there may be some cases where it is necessary to pay a small fee for certain special types of nodes.

Code Optimization

The code optimization in the synthesis phase is a program transformation technique, which tries to improve the intermediate code by making it consume fewer resources (i.e. CPU, Memory) so that faster-running machine code will result. Compiler optimizing process should meet the following objectives:

- The optimization must be correct, it must not, in any way, change the meaning of the program.
- Optimization should increase the speed and performance of the program.
- The compilation time must be kept reasonable.
- The optimization process should not delay the overall compiling process.

When to Optimize?

Optimization of the code is often performed at the end of the development stage since it reduces readability and adds code that is used to increase the performance.

Why Optimize?

Optimizing an algorithm is beyond the scope of the code optimization phase. So the program is optimized. And it may involve reducing the size of the code. So optimization helps to:

- Reduce the space consumed and increases the speed of compilation.
- Manually analysing datasets involves a lot of time. Hence we make use of software like Tableau for data analysis. Similarly, manually performing the optimization is also tedious and is better done using a code optimizer.
- An optimized code often promotes re-usability.

Types of Code Optimization –The optimization process can be broadly classified into two types:

1. **Machine Independent Optimization** – This code optimization phase attempts to improve the **intermediate code** to get a better target code as the output. The part of the intermediate code which is transformed here does not involve any CPU registers or absolute memory locations.

2. **Machine Dependent Optimization** – Machine-dependent optimization is done after the **target code** has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum **advantage** of the memory hierarchy.

Code Optimization is done in the following different ways:

1. Compile Time Evaluation:

(i) A = 2*(22.0/7.0) *r

Perform 2*(22.0/7.0)*r at compile time.

(ii) x = 12.4

y = x/2.3

Evaluate x/2.3 as 12.4/2.3 at compile time.

2. Variable Propagation :

//Before Optimization

c = a * b

x = a

till

d = x * b + 4

//After Optimization

c = a * b

x = a

till

d = a * b + 4

3.Common Sub-Expression Elimination-

The expression that has been already computed before and appears again in the code for computation is called as **Common Sub-Expression**.

In this technique,

- As the name suggests, it involves eliminating the common sub expressions.
- The redundant expressions are eliminated to avoid their re-computation.
- The already computed result is used in the further program when required.

Example-

Code Before Optimization	Code After Optimization
S1 = 4 x I S2 = a[S1] S3 = 4 x j S4 = 4 x i // **Redundant Expression** S5 = n S6 = b[S4] + S5	S1 = 4 x i S2 = a[S1] S3 = 4 x j S5 = n S6 = b[S1] + S5

4. Code Movement-

In this technique,

- As the name suggests, it involves movement of the code.
- The code present inside the loop is moved out if it does not matter whether it is present inside or outside.
- Such a code unnecessarily gets execute again and again with each iteration of the loop.
- This leads to the wastage of time at run time.

Example-

Code Before Optimization	Code After Optimization

for (int j = 0 ; j < n ; j ++)
{
x = y + z ;
a[j] = 6 x j;
}
	x = y + z ;
	for (int j = 0 ; j < n ; j ++)
	{
	a[j] = 6 x j;
	}

5. Dead Code Elimination-

In this technique,

- As the name suggests, it involves eliminating the dead code.
- The statements of the code which either never executes or are unreachable or their output is never used are eliminated.

Example-

Code Before Optimization	Code After Optimization
i = 0 ; if (i == 1) { a = x + 5 ;	i = 0 ;

}	

6. Strength Reduction-

In this technique,

- As the name suggests, it involves reducing the strength of expressions.
- This technique replaces the expensive and costly operators with the simple and cheaper ones.

Example-

Code Before Optimization	Code After Optimization
B = A x 2	B = A + A

Here,

- The expression "A x 2" is replaced with the expression "A + A".
- This is because the cost of multiplication operator is higher than that of addition operator.

LECTURE NO: 3
SOURCES OF OPTIMIZATION

Loop Optimization in Compiler Design

Loop Optimization is the process of increasing execution speed and reducing the overheads associated with loops. It plays an important role in improving cache performance and making effective use of parallel processing capabilities. Most execution time of a scientific program is spent on loops.

Loop Optimization is a machine independent optimization.

Decreasing the number of instructions in an inner loop improves the running time of a program even if the amount of code outside that loop is increased.

Loop Optimization Techniques:

1. **Frequency Reduction (Code Motion):**
   In frequency reduction, the amount of code in loop is decreased. A statement or expression, which can be moved outside the loop body without affecting the semantics of the program, is moved outside the loop.

   Example: Initial code:

```
while(i<100)
{
a = Sin(x)/Cos(x) + i;
i++;
}
```

   Optimized code:

```
t = Sin(x)/Cos(x);
while(i<100)
{
a = t + i;
i++;
}
```

2. **Loop Unrolling:**

Loop unrolling is a loop transformation technique that helps to optimize the execution time of a program. We basically remove or reduce iterations. Loop unrolling increases the program's speed by eliminating loop control instruction and loop test instructions.

Example:
Initial code:

```
for (int i=0; i<5; i++)
printf("Pankaj\n");
```

Optimized code:

```
printf("Pankaj\n");
printf("Pankaj\n");
printf("Pankaj\n");
printf("Pankaj\n");
printf("Pankaj\n");
```

3. **Loop Jamming:**
Loop jamming is the combining the two or more loops in a single loop. It reduces the time taken to compile the many number of loops.

Example:
Initial Code:

```
for(int i=0; i<5; i++)
a = i + 5;
for(int i=0; i<5; i++)
b = i + 10;
```

Optimized code:

```
for(int i=0; i<5; i++)
{
a = i + 5;
b = i + 10;
}
```

LECTURE NO:4
IDEA ABOUT GLOBAL DATA FLOW ANALYSIS

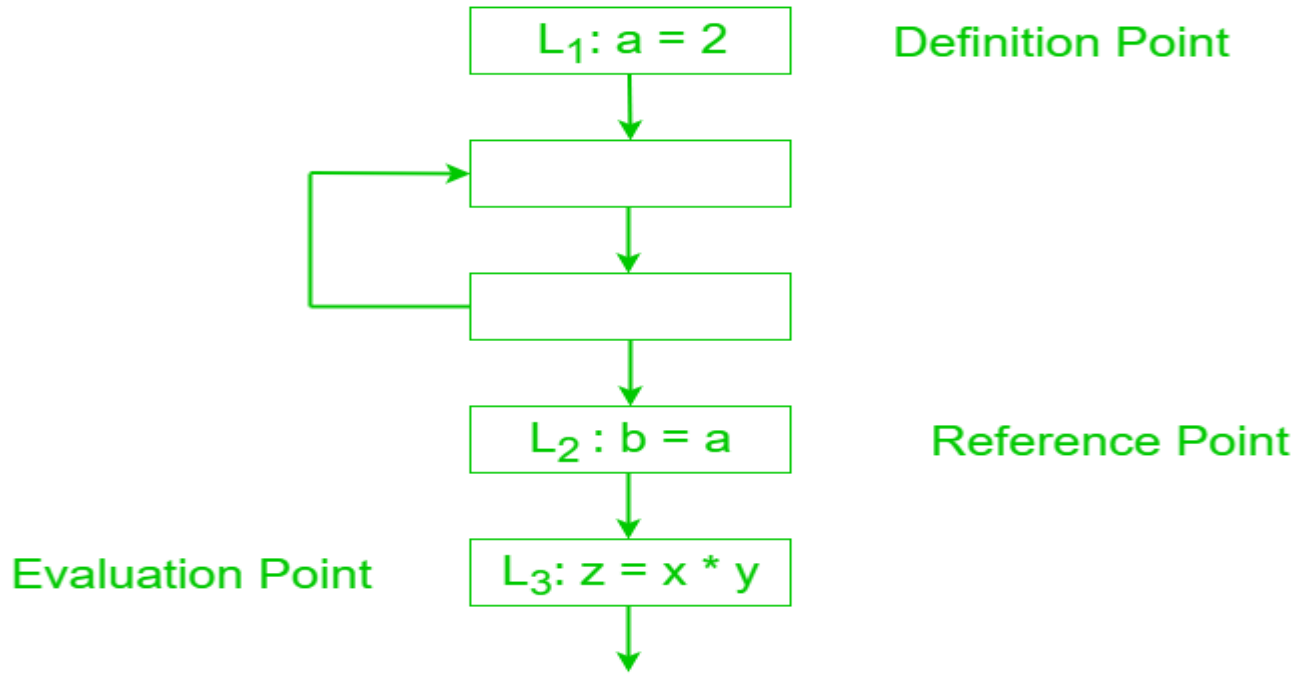
Data flow analysis in Compiler

It is the analysis of flow of data in control flow graph, i.e., the analysis that determines the information regarding the definition and use of data in program.

With the help of this analysis optimization can be done. In general, its process in which values are computed using data flow analysis. The data flow property represents information which can be used for optimization.

Basic Terminologies –

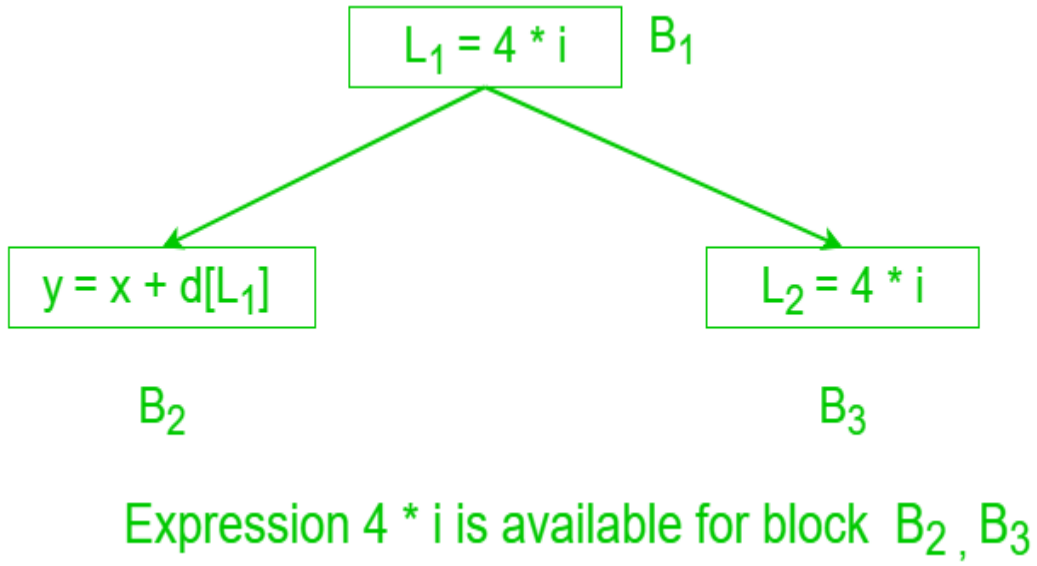
- **Definition Point:** a point in a program containing some definition.
- **Reference Point:** a point in a program containing a reference to a data item.
- **Evaluation Point:** a point in a program containing evaluation of expression.



Data Flow Properties –

- **Available Expression –** A expression is said to be available at a program point x iff along paths its reaching to x. A Expression is available at its evaluation point. A expression a+b is said to be available if none of the operands gets modified before their use.

Example –

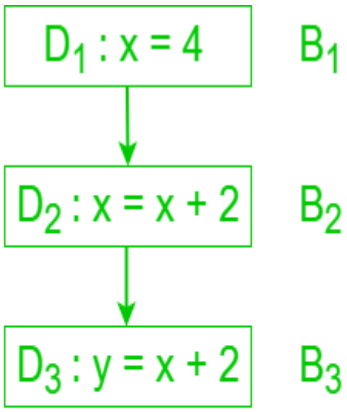


Advantage –

It is used to eliminate common sub expressions.

- **Reaching Definition –** A definition D is reaches a point x if there is path from D to x in which D is not killed, i.e., not redefined.

Example–



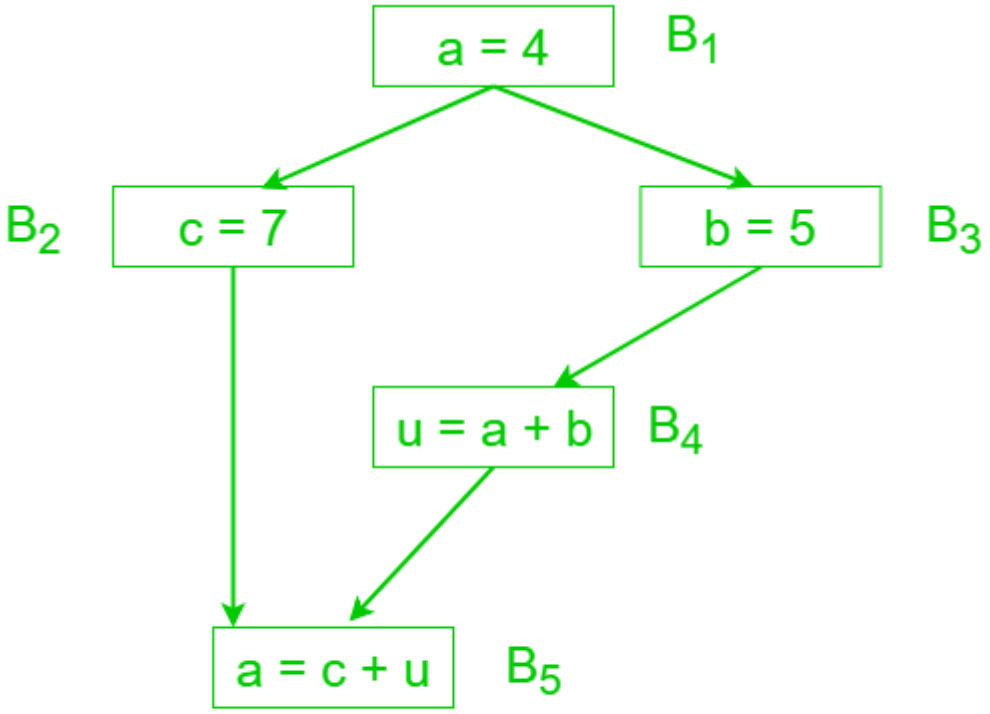
D_1 is reaching definition for B_2 but not for B_3 since it is killed by D_2

Advantage–

It is used in constant and variable propagation.

- **Live variable –** A variable is said to be live at some point p if from p to end the variable is used before it is redefined else it becomes dead.

Example –



a is live at block B_1 , B_3 , B_4 but killed at B_5

Advantage –

1. It is useful for register allocation.
2. It is used in dead code elimination.

- **Busy Expression –** An expression is busy along a path iff its evaluation exists along that path and none of its operand definition exists before its evaluation along the path.

Advantage –

It is used for performing code movement optimization.

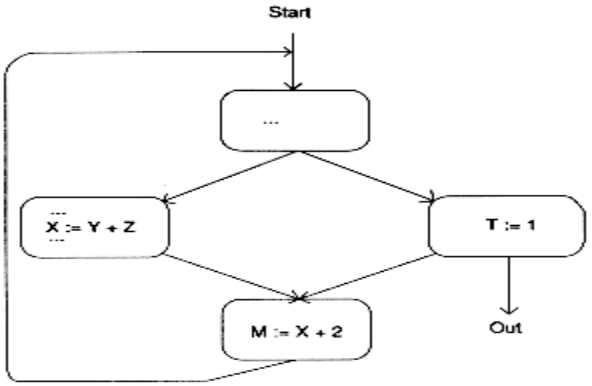
LOOP INVARIANT COMPUTATION

Loop-Invariant Computations and Code Motion

Loop-invariant statements are those statements within a loop which produce the same value each time the loop is executed. We identify them and then move them outside the loop.

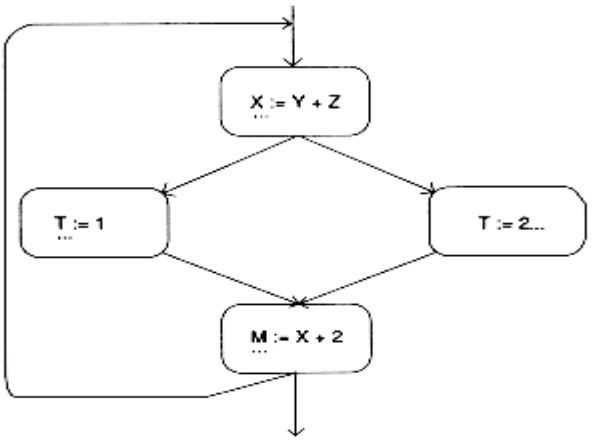
Example shows a control flow graph and a loop-invariant statement.

EXAMPLE 7 Loop invariants



In Example 7, *T := 1* is loop-invariant (presuming no assignment to *T* in the header) and may be moved outside the loop. The statement *X := Y + Z* is not loop invariant (why?).

EXAMPLE 8 Finding loop invariants



The definitions of *Y* and *Z* reaching *X := Y + Z* are outside the loop. This statement is then invariant. On a second pass, *M := X + 2* can be found to be loop-invariant. Once loop-invariant statements have been found, they are moved. One way to do this is to create a *preheader*.

Pre-headers

For each loop, create a new block, which has only the header as a successor. This is called a *pre-header* and is not part of the loop. It is used to hold statements moved out of the loop. Here, these are the loop-invariant statements. In the next section, the pre-header will be used to hold initializations for the induction variables.

LECTURE NO:5

PEEPHOLE OPTIMIZATION IN COMPILER DESIGN

Peephole Optimization

Peephole optimization is a type of Code Optimization performed on a small part of the code. It is performed on the very small set of instructions in a segment of code.

*The small set of instructions or small part of code on which peephole optimization is perform ed is known as **peephole** or **window**.*

It basically works on the theory of replacement in which a part of code is replaced by shorter and faster code without change in output.

Peephole is the machine dependent optimization.

Objectives of Peephole Optimization:

The objective of peephole optimization is:

1. To improve performance
2. To reduce memory footprint
3. To reduce code size

Peephole Optimization Techniques:

1. **Redundant load and store elimination:**
   In this technique the redundancy is eliminated.

2. **Initial code:**
3. y = x + 5;
4. i = y;
5. z = i;
6. w = z * 3;
7.
8. **Optimized code:**
9. y = x + 5;
10. i = y;
    w = y * 3;
11. **Constant folding:**
    The code that can be simplified by user itself, is simplified.

12. **Initial code:**
13. x = 2 * 3;
14.
15. **Optimized code:**

    x = 6;
16. **Strength Reduction:**
    The operators that consume higher execution time are replaced by the operators consuming less execution time.

17. **Initial code:**
18. y = x * 2;
19.
20. **Optimized code:**
21. y = x + x; or y = x << 1;
22.
23. **Initial code:**
24. y = x / 2;
25.
26. **Optimized code:**
    y = x >> 1;
27. **Null sequences:**
    Useless operations are deleted.
28. **Combine operations:**
    Several operations are replaced by a single equivalent operation.

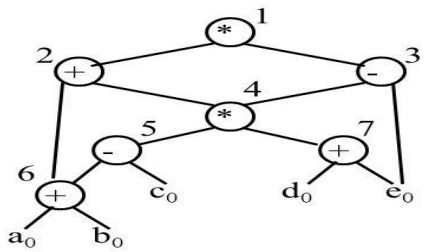
LECTURE NO: 6

CODE GENERATION FROM DAG

RULES FOR GENERATION DAG FROM CODE

1. A tree structure such that nodes may have more than one parent
2. Multiple parents are allowed so that an expression can be used more than once

An Example

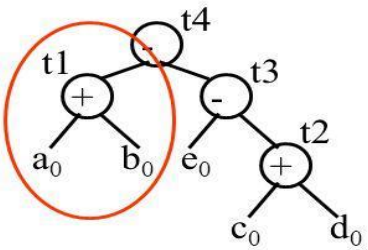



```
t7 := d + e
t6 := a + b
t5 := t6 - c
t4 := t5 * t7
t3 := t4 - e
t2 := t6 + t4
t1 := t2 * t3
```

57

Generating Code From DAGs

```
t1 := a + b
t2 := c + d
t3 := e - t2
t4 := t1 - t3
```



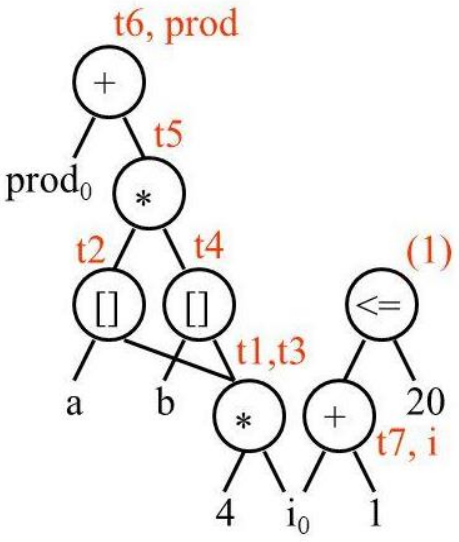
Only R0 and R1 available

```
(1)   MOV   a, R0
(2)   ADD   b, R0
(3)   MOV   c, R1
(4)   ADD   d, R1
(5)   MOV   R0, t1
(6)   MOV   e, R0
(7)   SUB   R1, R0
(8)   MOV   t1, R1
(9)   SUB   R0, R1
(10)  MOV   R1, t4
```

39

An Example

```
(1)    t1 := 4 * i
(2)    t2 := a[t1]
(3)    t3 := 4 * i
(4)    t4 := b[t3]
(5)    t5 := t2 * t4
(6)    t6 := prod + t5
(7)    prod := t6
(8)    t7 := i + 1
(9)    i := t7
(10)   if i <= 20 goto (1)
```



A SIMPLE CODE GENERATOR

- A code generator generates target code for a sequence of three- address statements and effectively uses registers to store operands of the statements.
- For example: consider the three-address statement

a = b+c It can have the following sequence of codes:

ADD R_j, R_i Cost = 1 // if R_i contains b and R_j contains c

(or)

ADD c, R_i Cost = 2 // if c is in a memory location

(or)

MOV c, R_j Cost = 3 // move c from memory to Rj and add

ADD R_j, R_i

Register and Address Descriptors:

- A register descriptor is used to keep track of what is currently in each registers. The register descriptors show that initially all the registers are empty.
- An address descriptor stores the location where the current value of the name can be found at run time.

A code-generation algorithm:

The algorithm takes as input a sequence of three -address statements constituting a basic block. For each three-address statement of the form x : = y op z**,** perform the following actions:

1. Invoke a function *getreg* to determine the location L where the result of the computation y op z should be stored.

2. Consult the address descriptor for y to determine y', the current location of y. Prefer the register for y' if the value of y is currently both in memory and a register. If the value of y is not already in L, generate the instruction **MOV y' , L** to place a copy of y in L.

3. Generate the instruction **OP z' , L** where z' is a current location of z. Prefer a register to a memory location if z is in both. Update the address descriptor of x to indicate that x is in location L. If x is in L, update its descriptor and remove x from all other descriptors.

4. If the current values of y or z have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of x : = y op z , those registers will no longer contain y or z.

Generating Code for Assignment Statements:

- The assignment d : = (a-b) + (a-c) + (a-c) might be translated into the following three-address code sequence:

```
t : = a – b
u : = a – c
v : = t + u
d : = v + u
```

with d live at the end.

Code sequence for the example is:

Statements	Code Generated	Register descriptor	Address descriptor
		Register empty	
t : = a – b	MOV a, R_0 SUB b, R0	R_0 contains t	t in R_0
u : = a – c	MOV a , R1 SUB c , R1	R_0 contains t R1 contains u	t in R_0 u in R1
v : =t + u	ADD R_1, R_0	R_0 contains v R_1 contains u	u in R_1 v in R_0
d : = v + u	ADD R_1, R_0 MOV R_0, d	R_0 contains d	d in R_0 d in R_0 and memory

Generating Code for Indexed Assignments

The table shows the code sequences generated for the indexed assignment statements **a : = b [i]** and **a [i] : = b**

Statements	Code Generated	Cost
a : = b[i]	MOV b(R_i), R	2
a[i] : = b	MOV b, a(R_i)	3

Generating Code for Pointer Assignments

The table shows the code sequences generated for the pointer assignments **a : = *p** and ***p : = a**

Statements	Code Generated	Cost
a : = *p	MOV *R_p, a	2
*p : = a	MOV a, *R_p	2

Issues in the Design of a Code Generator:

The following issues arise during the code generation phase:

1. Input to code generator
2. Target program
3. Memory management
4. Instruction selection
5. Register allocation
6. Evaluation order

Input to code generator

The input to code generator is the intermediate code generated by the front end, along with information in the symbol table that determines the run-time addresses of the data-objects denoted by the names in the intermediate representation. Intermediate codes may be represented mostly in quadruples, triples, indirect triples, Postfix notation, syntax trees, DAG's, etc. The code generation phase just proceeds on an assumption that the input are free from all of syntactic and state semantic errors, the necessary type checking has taken place and the type-conversion operators have been inserted wherever necessary.

Target program

The target program is the output of the code generator. The output may be absolute machine language, relocatable machine language, assembly language.

- Absolute machine language as output has advantages that it can be placed in a fixed memory location and can be immediately executed.
- Relocatable machine language as an output allows subprograms and subroutines to be compiled separately. Relocatable object modules can be linked together and loaded by linking loader. But there is added expense of linking and loading.
- Assembly language as output makes the code generation easier. We can generate symbolic instructions and use macro-facilities of assembler in generating code. And we need an additional assembly step after code generation.

Memory Management

Mapping the names in the source program to the addresses of data objects is done by the front end and the code generator. A name in the three address statements refers to the symbol table

entry for name. Then from the symbol table entry, a relative address can be determined for the name.

Instruction selection

Selecting the best instructions will improve the efficiency of the program. It includes the instructions that should be complete and uniform. Instruction speeds and machine idioms also plays a major role when efficiency is considered. But if we do not care about the efficiency of the target program then instruction selection is straight-forward.

```
P:=Q+R
S:=P+T
MOV Q, R0
ADD R, R0
MOV R0, P
MOV P, R0
ADD T, R0
MOV R0, S
```

Here the fourth statement is redundant as the value of the P is loaded again in that statement that just has been stored in the previous statement. It leads to an inefficient code sequence. A given intermediate representation can be translated into many code sequences, with significant cost differences between the different implementations.

Register allocation issues

Use of registers make the computations faster in comparison to that of memory, so efficient utilization of registers is important. The use of registers are subdivided into two subproblems:

- During **Register allocation –** we select only those set of variables that will reside in the registers at each point in the program.
- During a subsequent **Register assignment** phase, the specific register is picked to access the variable.

M a, b

These types of multiplicative instruction involve register pairs where a, the multiplicand is an even register and b, the multiplier is the odd register of the even/odd register pair.

Evaluation order

The code generator decides the order in which the instruction will be executed. The order of computations affects the efficiency of the target code. Among many computational orders, some will require only fewer registers to hold the intermediate results. However, picking the best order in the general case is a difficult NP-complete problem.